

Excer Tally Agent

A small program that lets the Excer website and TallyPrime talk to each other — orders flow into Tally automatically, stock flows back out. This explains what it is, how it works, and what still has to happen before it can be switched on.

16 September 2026 Forked from tally-prime-mcp (MIT) ~1,931 lines Recorded as CLAUDE.md §22

THE PROBLEM

Tally sits on a desk, not on the internet

Tally is a Windows program running on a computer in the client's office. It is not a cloud service. Two facts about it shape every decision below:

It never calls out

Tally has no webhooks and no notifications. It will answer when asked, but it will never tell you something changed. Anything resembling “live” has to be built by asking it, over and over.

It cannot be reached from outside

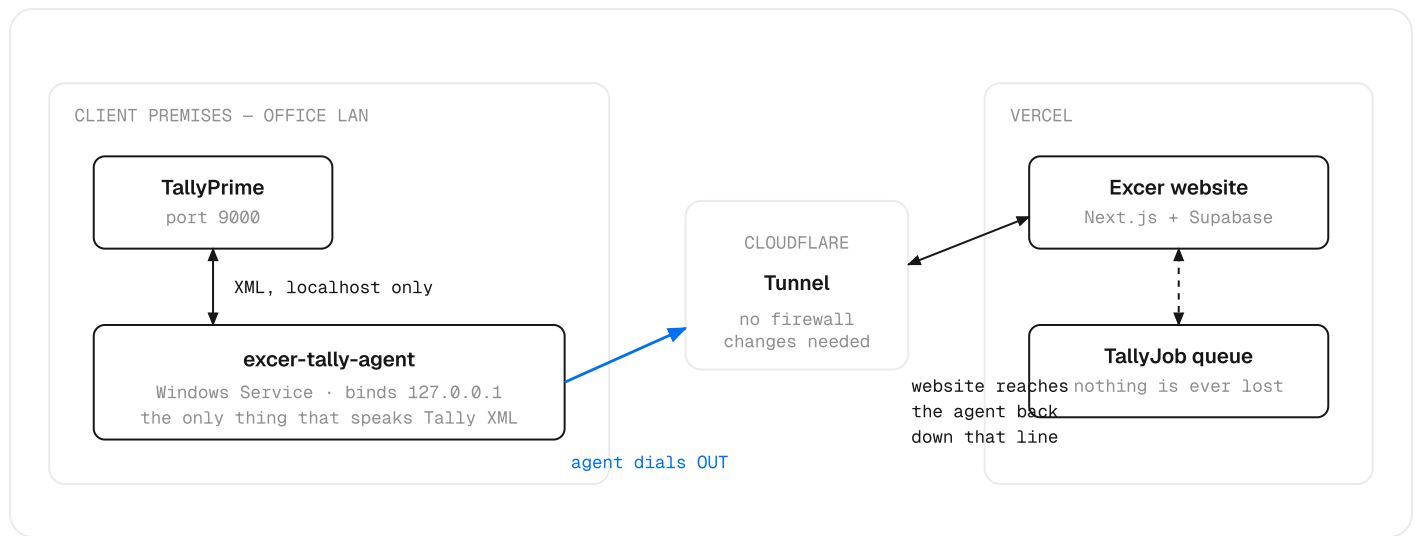
Its only doorway is port 9000 on the office network, and that doorway has **no password and no encryption**. Anyone who can reach it can rewrite the entire accounting books.

So the website cannot talk to Tally directly, and Tally cannot talk to the website at all. Something has to sit in the middle, inside the office, and do the talking.

WHAT WE BUILT

A translator that lives next to Tally

The agent runs on the office computer, beside Tally. The website speaks JSON; Tally speaks its own XML dialect. The agent converts between them — and it is the only piece in the whole system that knows any Tally XML at all.

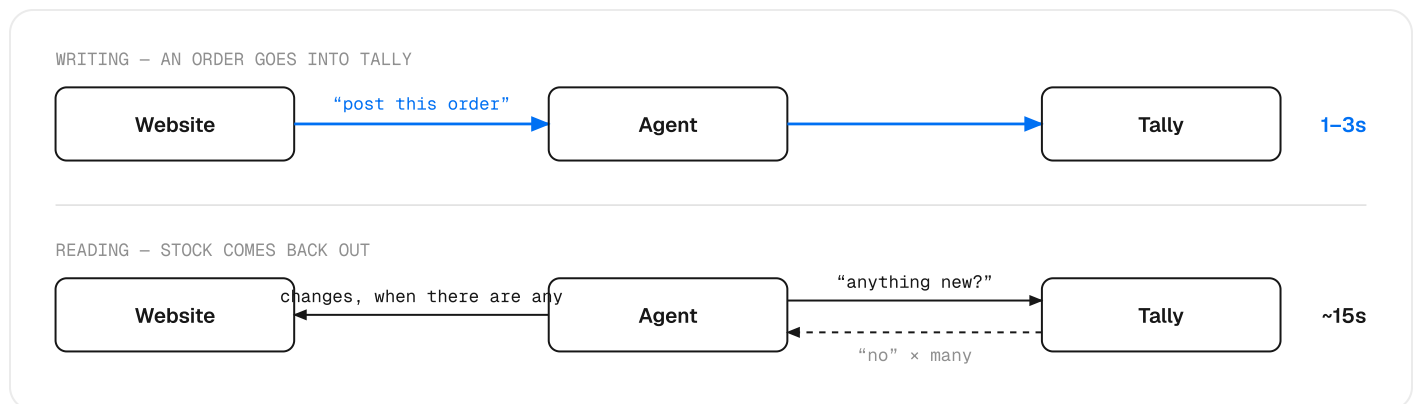


The office opens the connection outward, the way a browser does. Cloudflare then routes the website's requests back down that already-open line. Nothing is exposed, no firewall rule is added, and the client's IT department has nothing to approve.

HOW FAST IT IS

Fast one way, patient the other

Because Tally can be called but can never call, the two directions behave differently. This asymmetry is permanent — it is how Tally is built, and no product or vendor can remove it.



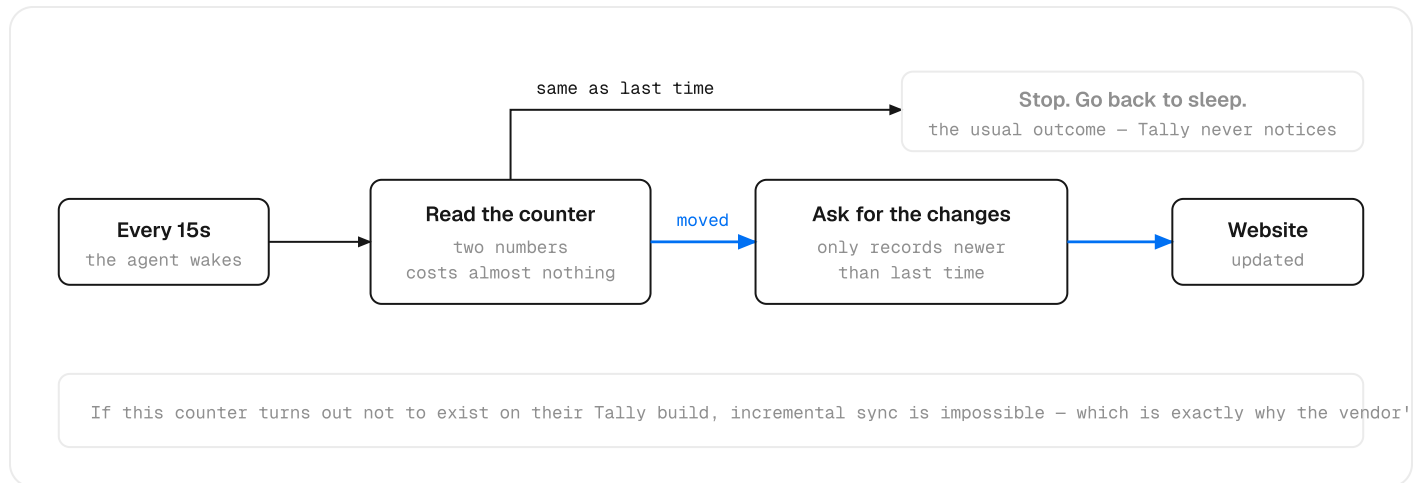
Orders reach Tally in seconds because the website starts the conversation. Stock updates lag by up to one polling cycle because the agent has to keep asking. Fifteen seconds is a setting, not a limit — it is simply the point where being current stops being worth the interruption.

THE KEY MECHANISM

Asking every 15 seconds without slowing Tally down

Tally does one thing at a time. A large request freezes the screen for whoever is using it, so polling it hard every fifteen seconds would make the accountant's day visibly worse — and we would be blamed for it.

The fix is to ask a **cheap question first**. Tally keeps a counter that ticks up whenever any record is edited. Reading that counter is nearly free. Only when the number has moved do we ask the expensive question, and even then we ask only for the records that changed.

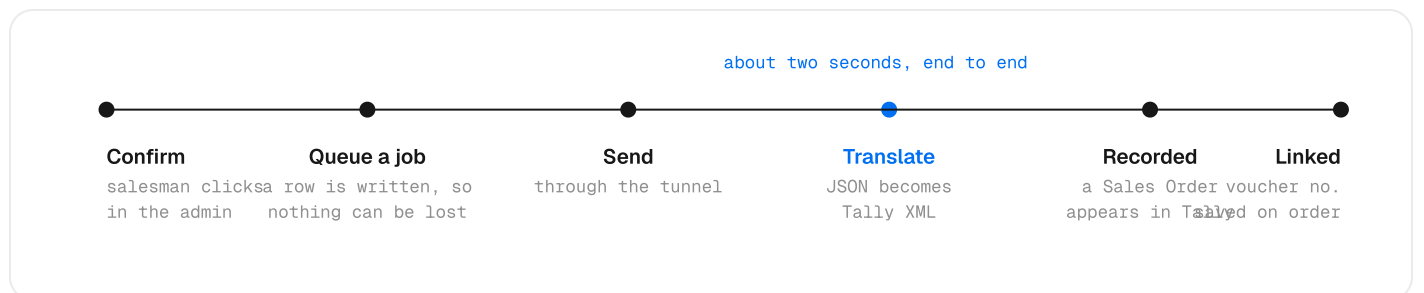


Two stages, and the first one answers “nothing changed” almost every time. This is what makes a fifteen-second rhythm neighbourly rather than hostile to the person actually using Tally.

Why we stopped waiting for the connector vendor. Their sample data had no change counter in it. Without one you cannot ask what changed — you can only re-download everything, on a button press, forever. That is roughly what the system does today, and it is why real-time was never reachable through them.

A DAY IN THE LIFE

What actually happens when an order is confirmed



The accountant opens Tally and the order is simply there. Nobody types it twice. The same shape covers dispatch, returns, cable cuts and cancellations — six document types in total.

If something goes wrong halfway

Every document carries a unique stamp. If the network drops and the same order is sent twice, Tally recognises the stamp and ignores the second copy. Without that, one bad moment on the line would put

two identical sales orders into a real client's books.

WHAT IT IS MADE OF

Roughly half of it we did not write

We started from an open-source project that already knew how to form Tally's XML, kept the parts worth keeping, deleted the rest, and added what was missing. Forking rather than depending on it means a quiet or abandoned upstream cannot affect us — we own the copy.

PART	WHERE IT CAME FROM	WHAT IT DOES
XML plumbing	Inherited, unchanged	Forms Tally's envelopes, dates and escaping — the tedious, easy-to-get-subtly-wrong half
Document rendering	Inherited, adapted	Builds vouchers; we added the duplicate-protection stamp
The six documents	New	Sales order, delivery note, credit note, customer, stock journal, cancellation
Change detection	New	The two-stage counter check
Web server	New	The two doors the website knocks on
Heartbeat	New	Reports “I am alive” so a red light can appear in the admin panel
Doctor	New	Checks whether Tally answers correctly — run before anything else
Installer	New	One command to set it all up on the office PC

USING IT

Day to day, nobody touches it

That is the whole point. Once installed it runs in the background, starts when the computer does, and restarts itself if it crashes. There is no screen to open and no button to press.

ORDER CONFIRMED

~2s

appears in Tally

STOCK EDITED IN TALLY

~15s

website catches up

STAFF EFFORT

None

no double entry

Setting it up — once, about 45 minutes

- 1 **Turn on Tally's connection setting.** One checkbox inside Tally, then restart it.
- 2 **Install Node.js.** A normal Windows installer.
- 3 **Copy the agent onto the PC.** Somewhere permanent — not a Desktop, not a synced folder.
- 4 **Fill in the settings file.** Company name, a password we generate, and the client's own Tally naming.
- 5 **Run the doctor.** Checks Tally answers properly. **Do not skip this** — it is where wrong assumptions surface while you are still standing there.
- 6 **Run the installer.** One command. Makes it run forever, restarting on failure.
- 7 **Connect the tunnel.** Links it to the website without opening the firewall.

HONEST STATUS

What is done, and what is not

• BUILT

The agent itself

Written, type-checked, builds cleanly, starts and stops properly, rejects bad requests, and reports clear errors when Tally is unreachable. Committed, with an install script and a written runbook.

● UNVERIFIED **It has never met a real Tally**

Every Tally installation names things differently — voucher types, ledgers, where the tax rate is stored. Right now those are educated guesses. Six specific assumptions are listed in the code and must be checked on the client's machine. The doctor is what checks them.

● OPEN **One deliberate blank — the tax rule**

A Kerala customer is taxed CGST + SGST; an out-of-state customer is taxed IGST. That is about eight lines of code, but it is a business decision that misfiles tax in a real client's books if it is wrong — so it was left to be decided, not guessed.

● BLOCKED **Two questions for the client**

Neither is a coding problem, and both outrank everything above.

- **May we install a program on the Tally computer?** If the answer is no, the whole approach needs rethinking — so ask first.
- **When the website and Tally disagree about stock, which one is right?** This sounds minor. It is not. Get it wrong and the same goods are deducted twice, once by each system.

SET EXPECTATIONS

What the client should be told plainly

- **The office computer must be on, with Tally open.** Orders placed while it is off are queued and go out when it returns — nothing is lost, but nothing is instant either.
- **Website stock is about fifteen seconds behind Tally.** Orders going into Tally are near instant.
- **If that computer is replaced or rebuilt, we need to know.** The agent has to be reinstalled.
- **Nothing goes into the live books first.** The first documents are posted into a test company, and only then switched over.

Excer Global · Tally integration handover · 16 September 2026

Full technical record in `CLAUDE.md §22` . Install runbook in `install/INSTALL.md` . Agent repository: `excer-tally-agent` .